

OpenMP 多线程编程

——以高斯消去为例

曾泉胜

2022 年 5 月

陈静怡

2023 年 4 月

唐明昊

2024 年 4 月

华志远，张逸非，孔德嵘

2025 年 5 月

邹博闻

2026 年 5 月

目录

1 实验介绍	3
1.1 实验选题	3
1.2 实验要求	3
2 实验设计指导	3
2.1 OpenMP 简介	3
2.1.1 学习链接	3
2.1.2 基本语法	4
2.1.3 编程范式	5
2.1.4 更多细节	6
2.2 实验总体思路	7
2.3 编译、运行	8
3 ANN 选题：多线程实验	8
4 NTT 选题：多线程实验	8
5 口令猜测选题：多线程实验	8
6 矩阵 SVD 选题：多线程实验	8
7 Vtune 分析	9

1 实验介绍

有一句调侃的话叫做，“并行程序设计就是索引分配的艺术”。同学们在 pthread 实验之后，应该或多或少感受到了最简单的并行化方式就是把一个好的串行化算法分配给不同线程，按行、按列或者按块划分等等。重复性的工作必然会被自动化的工具代替，于是我们来学习一种很方便的工具：OpenMP。

1.1 实验选题

同 SIMD 编程作业。

1.2 实验要求

参看 pthread 作业指导书。

2 实验设计指导

2.1 OpenMP 简介

2.1.1 学习链接

OpenMP (Open Multi-Processing) 是一套支持跨平台共享内存方式的多线程并发的编程 API，使用 C、C++ 和 Fortran 语言，可以在大多数的处理器体系和操作系统中运行，包括 Solaris, AIX, HP-UX, GNU/Linux, Mac OS X, 和 Microsoft Windows。包括一套编译器指令、库和一些能够影响运行行为的环境变量。

首先需要有一个支持 OpenMP 的编译器，自 GCC 4.9 起，C、C++ 完全支持 OpenMP 4.0；自 GCC 6 起，C 和 C++ 完全支持 OpenMP 4.5。自 GCC 9 起，C 和 C++ 部分支持 OpenMP 5.0，并在 GCC 10 中得到扩展。更多编译器支持信息可见<https://www.openmp.org/resources/openmp-compilers-tools/>。

可以从官网获得文档 (<https://www.openmp.org/resources/refguides/>) 以及各种教程 (<https://www.openmp.org/resources/tutorials-articles/>)。课堂 PPT 中也给了大量的例子。

2.1.2 基本语法

OpenMP 的基本语法是将如下格式的预编译指令用在一个语句块之前：

```
#pragma omp <directive> [clause[, clause] ...]
```

其中，directive 可以是以下内容：

1. parallel: 创建线程，接下来的代码块是一个并行区域
2. single: 之后的代码块只会由一个线程（未必是主线程）执行
3. master: 之后的代码块只会由主线程执行
4. for: 之后的 for 循环将被并行化由多个线程划分执行，循环变量必须是整型
5. barrier: 所有线程在此同步
6. critical: 之后的代码块一次只有一个线程可以执行（临界区）
7. ...

clause 可以是以下内容：

1. if(*scalar-expression*) 当跟在 parallel 指令之后使用时表示是否并行化
2. private(*list*) 指定并行区域中的变量为每个线程独立的存储
3. shared(*list*) 指定并行区域中的变量为各个线程共享
4. default(shared|none) 未指定变量缺省的存储方式，shared 表示共享，none 表示必须为所有变量指定存储方式
5. nowait 在离开下方的语句块时不进行线程同步
6. schedule(static|dynamic|guided[, *chunk_size*]) 用在 for 指令之后，指定循环任务的划分方法。
7. ...

此外，OpenMP 还提供了一些库函数：

```
1 // 设置之后一个并行区域的线程数, 只能在串行代码区域使用
2 void omp_set_num_threads(int _Num_threads);
3
4 // 获得当前线程数目
5 int omp_get_num_threads(void);
6
7 // 返回当前线程id.id从1开始顺序编号,主线程id是0
8 int omp_get_thread_num(void);
9
10 // 获得当前墙上时钟时间, 每个线程都有自己的时间
11 double omp_get_wtime(void);
```

2.1.3 编程范式

图2.1展示了使用 OpenMP 程序的运行模式, 下面展示的是一种编程范式, 以普通高斯消去为例, 编写如下代码, 并在使用 gcc 编译时使用-fopenmp 选项:

```
1 // 需要包含头文件omp.h
2 #include <omp.h>
3 // ...
4 // 在外循环之外创建线程, 避免线程反复创建销毁, 注意共享变量和私有变量的设置
5 #pragma omp parallel if(parallel), num_threads(NUM_THREADS), private(i, j, k, tmp)
6 for(k = 1; k < n; ++k){
7     // 串行部分, 也可以尝试并行化
8     #pragma omp single
9     {
10         tmp = mat[k][k];
11         for(j = k + 1; j < n; ++j){
12             mat[k][j] = mat[k][j] / tmp;
13         }
14         mat[k][k] = 1.0;
15     }
16     // 并行部分, 使用行划分
17     #pragma omp for
18     for(i = k + 1; i < n; ++i){
19         tmp = mat[i][k];
20         for(j = k + 1; j < n; ++j){
21             mat[i][j] = mat[i][j] - tmp * mat[k][j];
22         }
23         mat[i][k] = 0.0;
24     }
25     // 离开for循环时, 各个线程默认同步, 进入下一行的处理
26 }
```

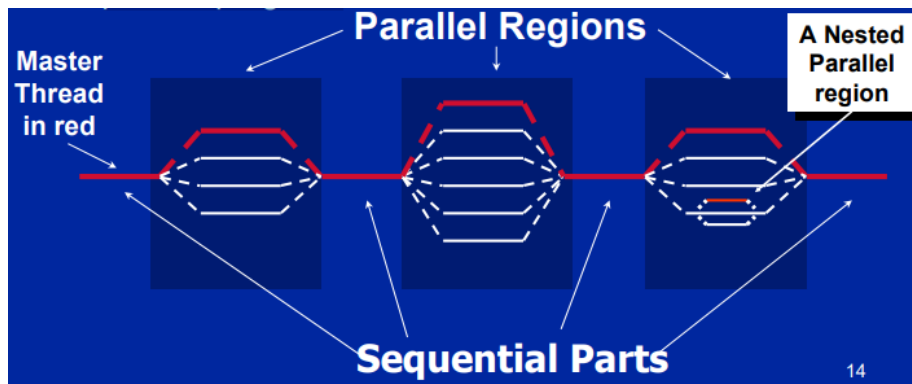


图 2.1: OpenMP 的 fork-join 并行机制, 参考<https://www.openmp.org/wp-content/uploads/omp-hands-on-SC08.pdf>

2.1.4 更多细节

OpenMP 为我们提供了方便的 API, 在此基础上, 可以根据研究的问题进行更多的探索, 在此简单罗列几个可以尝试的方向。

负载均衡问题 在为线程划分任务的时候, 需要避免负载不均, 以高斯消去为例, 对于倒数第 k 行, 消去过程的计算量为 $O(k^2)$, 使用纯块划分会使得处理头几行的线程负载要高于处理最后几行的线程, 在 OpenMP 的 schedule 子句默认的划分方式 (**static**) 就是这样, 可以通过设置 `chunk_size`, 使用循环划分或者块循环划分来均衡负载。

当研究的问题负载不均并且不像高斯消去这样可以预先知道负载的分布时, 可以使用动态的划分方式 (**dynamic**) 并设置一定的 `chunk_size`, 该方式下某个线程结束了先前的任务之后, 又会从任务队列中取出新的任务, 而并不依赖于线程号与行号的某种映射关系。**guided** 方式把循环体的执行分组, 分配给等待执行的线程。最初的组中的循环体执行数目较大, 然后逐渐按指数方式下降到 `chunk_size`。

以一个例子来帮助同学们理解 `chunk_size`: 如果你有 100 次迭代和 4 个线程, 那么默认情况下, 每个线程会处理 25 次迭代 (static)。但是, 如果你设置了 `chunk_size` 为 10, 那么每个线程会一次处理 10 次迭代, 按间隔再获取下一个 10 次迭代, 直到所有的迭代都被处理完。dynamic 方式下也是类似, 设置了 `chunk_size` 后会按大小从任务池拿取。

较小的 `chunk_size` 可以帮助平衡负载，避免某些线程早早完成它们的迭代而闲置，而其他线程还在处理它们的迭代。然而，过小的 `chunk_size` 也会增加线程之间的同步开销，因此需要根据具体的应用情况来选择合适的 `chunk_size`。

虚假共享问题 如果想要分析多线程程序的 cache，就必须认识到在现代多核处理器中，可能是每个核都有独自的一级和二级 cache，然后有所有核共享的三级 cache（具体情况应当使用相关指令查阅自己的机器），这样一来，cache 不命中的原因除了冷启动、容量不足或者发生冲突之外，还会因为核 B 试图写入核 A 的 cache 中存在的某个高速缓存行，使得核 A 中的该缓存行失效，下一次核 A 访问该缓存行不命中。该现象被称为虚假共享。我们可以使用在共享访问的变量间适当添加填充，对齐到高速缓存行的大小（例如 64B），或者在多线程访问连续内存时，适当提高 `chunk_size`，使得一个高速缓存行放在一个线程里解决。

使用 OpenMP 提供的归约操作 在高斯消去的例子中，运算基本都是矩阵到矩阵的方式，因此不会用到归约操作。但是例如求和，求最大值这样从矢量到标量的计算，先前提提供的范式不能得到正确的结果，而使用加锁等并发编程方式将大大增加计算开销，而 OpenMP 中为我们提供了归约操作，甚至还可以自定义归约的运算，需要的同学可以自行研究。

使用 SIMD 与 Pthread 实验一样，也可以在内层循环使用循环展开，SIMD 指令进一步加速；OpenMP 4.0 之后也提供了自动向量化的方式可以尝试。

嵌套的线程 我们知道在 `fork()` 中再次调用 `fork()`，可以得到 4 个一模一样的克隆，但是在缺省状态下，嵌套并行是禁用的，因此上文中在并行区域中再写 `#pragma omp parallel` 并不会使得线程数加倍，大多数情况下也并不推荐使用，容易造成资源的浪费。

2.2 实验总体思路

本次实验可以将上次 Pthread 的实验改写成 OpenMP 的方式，同样的可以在任务划分方式、与 SIMD 的结合，算法的复杂度分析（运行时间、

加速比、伸缩性分析), 进行 profiling 等等, 也可以将 OpenMP 的实现与 Pthread 的实现进行对比, 在保证结果正确的前提下尝试改进性能。

2.3 编译、运行

支持 OpenMP 的程序必须使用 `-fopenmp` 选项进行编译, 其他注意事项参见之前的实验教学指导书。

(先不用管) 在 Intel DevCloud 平台, 已部署新的用于 oneAPI 的编译器, 可替代旧的 Intel 编译器。编译 OpenMP (卸载) 的示例如下, 同学们可尝试。

```
1 icx -fopenmp -fopenmp-targets=spir64 -o saxpy saxpy.c
```

3 ANN 选题：多线程实验

对于 ANN 选题, openmp 的实现思路和 pthread 几乎相同, 算法细节参考 pthread 实验指导书即可。

4 NTT 选题：多线程实验

NTT 选题的 openmp 选题较为简单, 参考高斯消元的方法对进阶选题中的代码实现自动优化即可。

5 口令猜测选题：多线程实验

实现思路和 pthread 指导书中的介绍和要求完全相同, 请参照对应内容进行实现。

6 矩阵 SVD 选题：多线程实验

请参考 pthread 编程。需要额外注意的是, OpenMP 实现线程池可能较 pthread 复杂, 很可能自己写半天不如简单调用一句代码, 因此 OpenMP 版本的线程池不做强制要求, 只要求 pthread 版本的 (当然尝试去优化会有一定加分)。

7 Vtune 分析

此部分可见 Pthread 实验指导书。